RISC Zero 아키텍처 및 핵심 개념

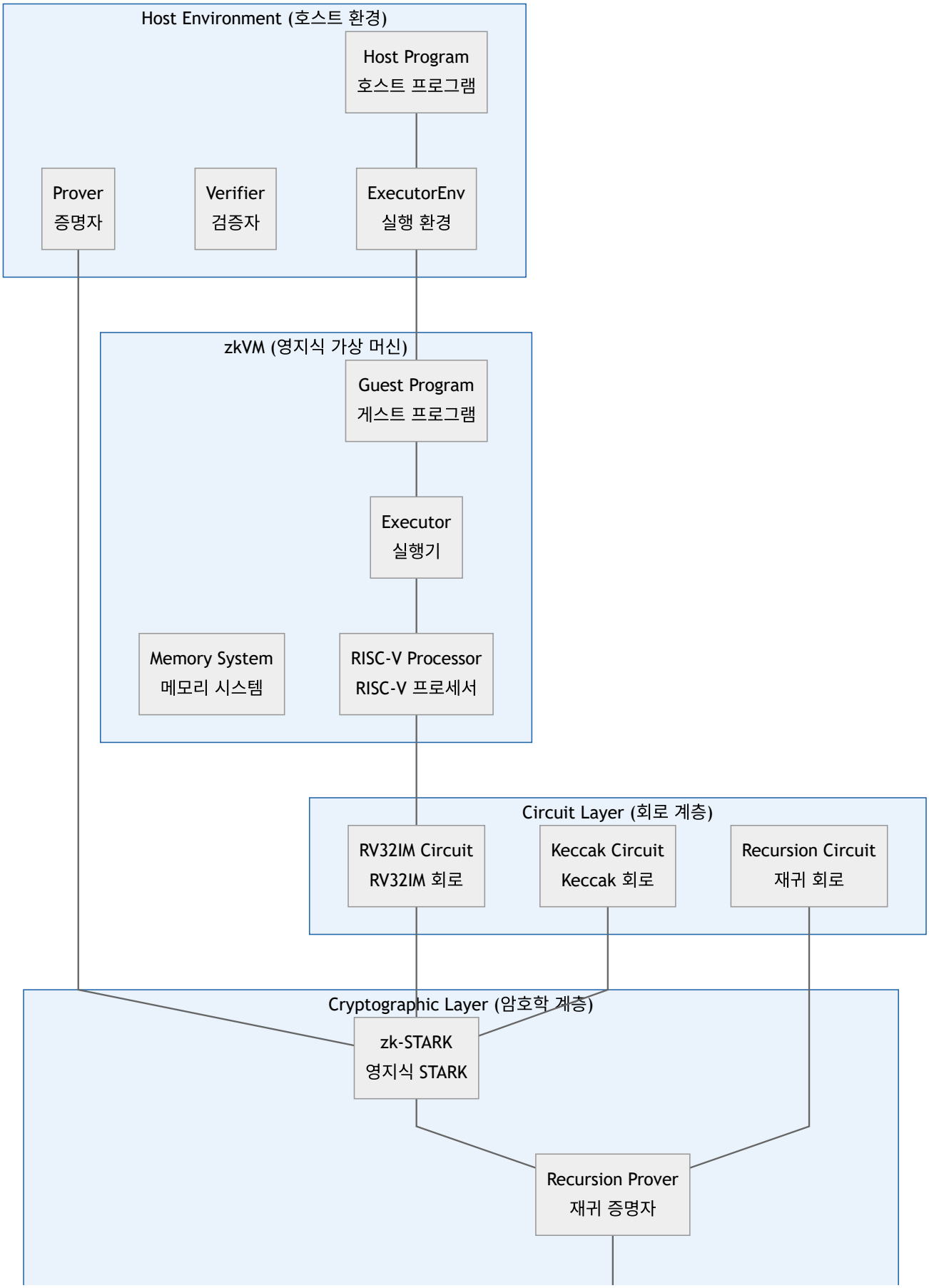
개요

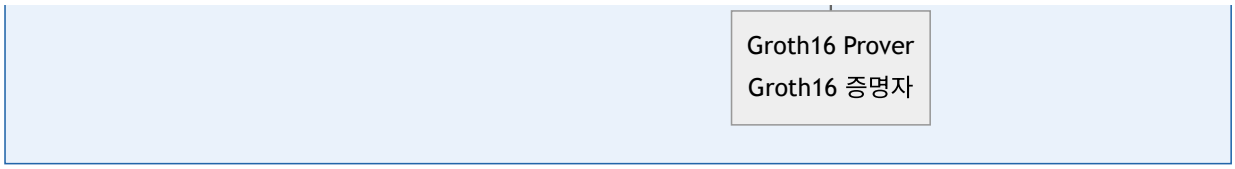
RISC Zero는 zk-STARK(Zero-Knowledge Scalable Transparent Arguments of Knowledge)와 RISC-V 마이크로아키텍처를 기반으로 한 영지식 증명 가능한 범용 컴퓨팅 플랫폼입니다. 이 시스템은 임의의 Rust 코드를 RISC-V 가상 머신에서 실행하고, 그 실행에 대한 암호학적 증명을 생성할 수 있습니다.

전체 시스템 아키텍처

주요 컴포넌트 구조

RISC Zero는 다음과 같은 계층적 구조로 구성됩니다:


Host Environment (호스트 환경)
Host Program
호스트 프로그램
Prover
증명자
Verifier
검증자
ExecutorEnv
실행 환경
zkVM (영지식 가상 머신)
Guest Program
게스트 프로그램
Executor
실행기
Memory System
메모리 시스템
RISC-V Processor
RISC-V 프로세서
Circuit Layer (회로 계층)
RV32IM Circuit
RV32IM 회로
Keccak Circuit
Keccak 회로
Recursion Circuit
재귀 회로
Cryptographic Layer (암호학 계층)
zk-STARK
영지식 STARK
Recursion Prover
재귀 증명자



핵심 크레이트 구조

RISC Zero는 다음과 같은 주요 크레이트들로 구성됩니다:

1. 사용자 인터페이스 계층

- **`risc0-zkvm`** : 메인 API 및 사용자 인터페이스 제공
- **`risc0-build`** : Guest 프로그램 빌드 시스템
- **`cargo-risczero`** : 개발 도구 및 프로젝트 템플릿

2. 실행 계층

- **`risc0-zkvm/host`** : Host 환경 구현 (증명 생성/검증)
- **`risc0-zkvm/guest`** : Guest 환경 API (I/O, 시스템 호출)
- **`risc0-r0vm`** : RISC-V 가상 머신 구현

3. 회로 계층

- **`risc0-circuit-rv32im`** : RISC-V RV32IM 명령어 집합 회로
- **`risc0-circuit-keccak`** : Keccak 해시 함수 회로
- **`risc0-circuit-recursion`** : 재귀적 증명을 위한 회로

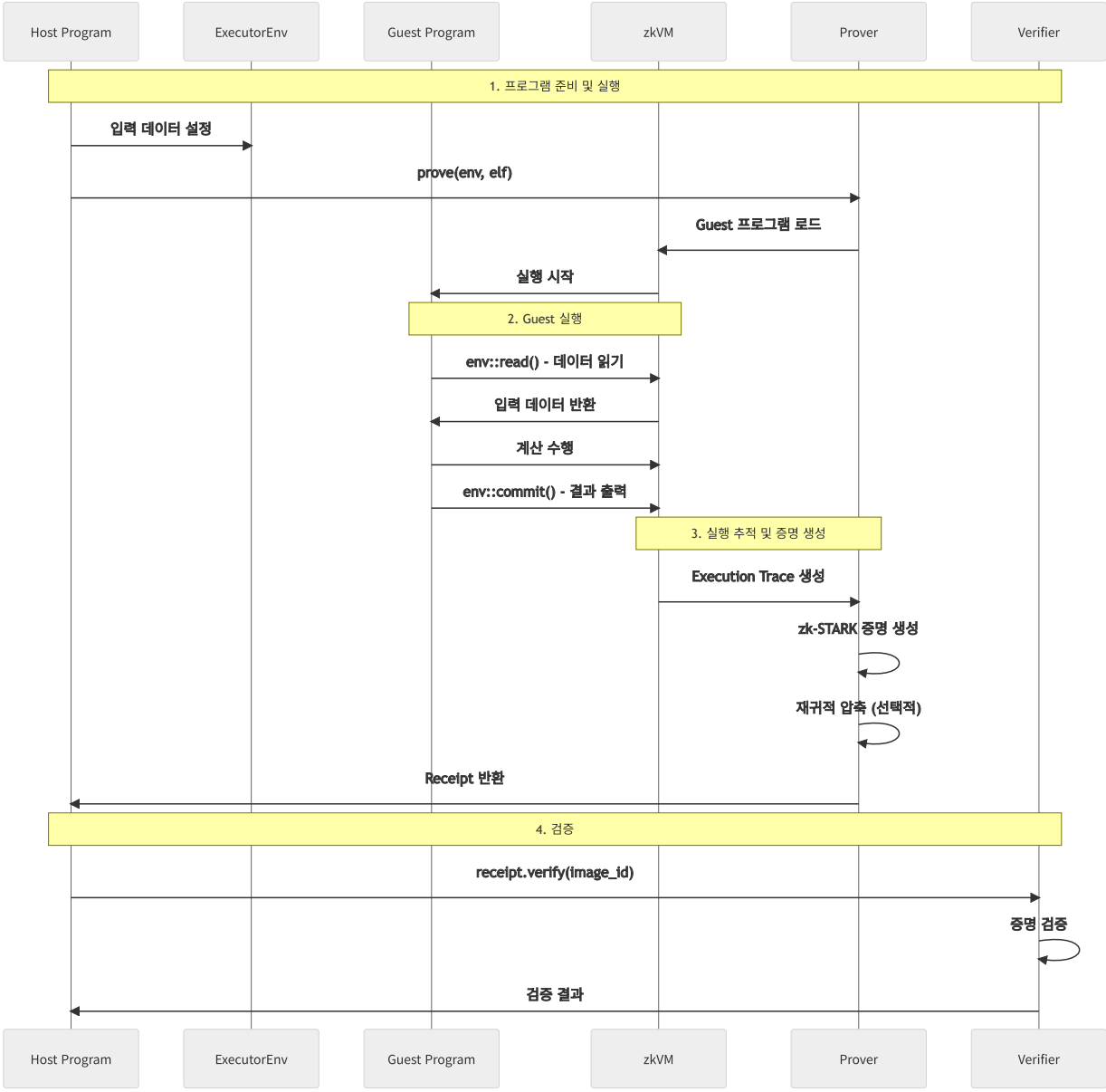
4. 암호학 계층

- **`risc0-zkp`** : zk-STARK 증명 시스템 구현

- `risc0-groth16` : Groth16 증명 시스템 (STARK → SNARK 압축)

데이터 흐름 다이어그램

전체 시스템의 데이터 흐름은 다음과 같습니다:


Host Program
ExecutorEnv
Guest Program
zkVM
Prover
Verifier
1. 프로그램 준비 및 실행
입력 데이터 설정
prove(env, elf)
Guest 프로그램 로드
실행 시작
2. Guest 실행
env::read() - 데이터 읽기
입력 데이터 반환
계산 수행
env::commit() - 결과 출력
3. 실행 추적 및 증명 생성
Execution Trace 생성
zk-STARK 증명 생성
재귀적 압축 (선택적)
Receipt 반환
4. 검증
receipt.verify(image_id)
증명 검증
검증 결과
Host Program
ExecutorEnv
Guest Program
zkVM
Prover
Verifier

zkVM, Host, Guest의 역할과 상호작용

Host (호스트)

역할: 증명 생성과 검증을 담당하는 신뢰할 수 있는 환경

주요 기능:

- Guest 프로그램에 입력 데이터 제공
- zkVM에서 Guest 프로그램 실행 요청
- 실행 결과에 대한 암호학적 증명 생성
- 생성된 Receipt 검증

핵심 API:

```
// 실행 환경 구성
let env = ExecutorEnv::builder()
.write(&input_data) // Guest에 데이터 전송
.build()?;


// 증명 생성
let prover = default_prover();
let receipt = prover.prove(env, GUEST_ELF)?.receipt;
```

```
// 검증
receipt.verify(IMAGE_ID)?;
```

Guest (게스트)

역할: zkVM 내에서 실행되는 프로그램으로, 증명하고자 하는 계산 로직 포함

주요 특징:

- `no_std` 환경에서 실행 (표준 라이브러리 제한)
- RISC-V 명령어로 컴파일됨
- Host와의 통신은 특별한 API를 통해서만 가능

핵심 API:

```
#![no_main]
#![no_std]

use risc0_zkvm::guest::env;

risc0_zkvm::guest::entry!(main);

fn main() {
// Host로부터 데이터 읽기
let input: u64 = env::read();

// 계산 수행
```

```
let result = compute(input);


// 결과를 Journal에 기록 (공개 출력)
env::commit(&result);
}
```

zkVM (영지식 가상 머신)

역할: Guest 프로그램을 실행하고 실행 과정을 추적하여 증명 생성을 위한 데이터 제공

주요 기능:

- RISC-V 명령어 해석 및 실행
- 메모리 관리 및 시스템 호출 처리
- 실행 추적(Execution Trace) 생성
- Host-Guest 간 I/O 중재

실행 과정:

1. Guest ELF 바이너리 로드
2. RISC-V 명령어 순차 실행
3. 각 명령어 실행 상태 기록
4. 메모리 접근 및 시스템 호출 추적
5. 실행 완료 후 Execution Trace 반환

핵심 개념 및 용어 정의

zk-STARK (Zero-Knowledge Scalable Transparent Arguments of Knowledge)

정의: 영지식 증명 시스템의 한 종류로, 확장 가능하고 투명한 특성을 가짐

RISC Zero에서의 적용:

- **확장성(Scalable)**: 계산 복잡도에 따라 증명 크기가 로그적으로 증가
- **투명성(Transparent)**: 신뢰할 수 있는 설정(trusted setup) 불필요
- **영지식(Zero-Knowledge)**: 계산 결과만 공개하고 입력은 비공개 유지

기술적 특징:

- FRI(Fast Reed-Solomon Interactive Oracle Proof) 프로토콜 사용
- 다항식 커밋먼트를 통한 증명 생성
- 약 98비트 보안 강도 제공

RISC-V 아키텍처

정의: 개방형 명령어 집합 아키텍처(ISA)로, RISC Zero의 기반이 되는 프로세서 아키텍처

RISC Zero에서의 구현:

- **RV32IM**: 32비트 기본 정수 명령어 + 곱셈/나눗셈 확장
- **메모리 모델**: 단일 주소 공간, 바이트 단위 주소 지정
- **레지스터**: 32개의 범용 레지스터 (x0-x31)

회로 구현:

```
// risc0-circuit-rv32im에서 구현되는 주요 구조
pub struct ExecutionTrace {
pub pc: Vec<u32>, // 프로그램 카운터
pub registers: Vec<[u32; 32]>, // 레지스터 상태
pub memory: Vec<MemoryOp>, // 메모리 연산
pub syscalls: Vec<SyscallOp>, // 시스템 호출
```

```
}
```

재귀적 증명 (Recursive Proofs)

정의: 기존 증명을 입력으로 받아 새로운 증명을 생성하는 기법

RISC Zero의 3계층 재귀 구조:

1. **STARK 계층**: 기본 실행 증명 생성
2. **Lift 계층**: STARK 증명을 재귀 회로로 변환
3. **Join 계층**: 여러 증명을 하나로 결합

장점:

- 증명 크기 압축 (수백 KB → 수백 바이트)
- 배치 처리를 통한 효율성 향상
- 복합 계산의 모듈화 가능

Method (메소드)

정의: Guest 프로그램이 컴파일된 RISC-V ELF 바이너리와 관련 메타데이터

구성 요소:

- **ELF 바이너리**: 실행 가능한 RISC-V 코드
- **Image ID**: 프로그램의 암호학적 해시 (무결성 보장)
- **메타데이터**: 메모리 레이아웃, 진입점 등

생성 과정:

```
// risc0-build를 통한 Method 생성
fn main() {
```

```
risc0_build::embed_methods();

}


// 생성된 상수들

pub const MULTIPLY_ELF: &[u8] = include_bytes!("...");

pub const MULTIPLY_ID: [u32; 8] = [0x12345678, ...];
```

Image ID (이미지 ID)

정의: Method의 고유 식별자로, 프로그램 코드의 암호학적 해시

특징:

- 256비트 (32바이트) 해시 값
- 프로그램 코드 변경 시 자동으로 변경됨
- 검증 시 Receipt와 함께 사용하여 프로그램 무결성 보장

용도:

- 증명 검증 시 올바른 프로그램이 실행되었는지 확인
- 프로그램 버전 관리 및 무결성 검증

Receipt (영수증)

정의: zkVM 실행 결과와 그에 대한 암호학적 증명을 포함하는 구조체

구조:

```
pub struct Receipt {

pub inner: InnerReceipt, // 실제 증명 데이터
```

```
pub journal: Journal, // 공개 출력 데이터
}

pub enum InnerReceipt {
Stark(StarkReceipt), // zk-STARK 증명
Groth16(Groth16Receipt), // Groth16 증명 (압축됨)
Fake(FakeReceipt), // 개발용 가짜 증명
}
```

검증 과정:

1. Image ID와 Receipt의 프로그램 해시 비교
2. 암호학적 증명 검증 (STARK 또는 Groth16)
3. Journal 데이터 무결성 확인

Journal (저널)

정의: Guest 프로그램이 `env::commit()` 을 통해 출력한 공개 데이터의 순차적 로그

특징:

- **순차적**: 데이터가 commit된 순서대로 저장
- **공개**: 누구나 읽을 수 있는 공개 정보
- **불변**: 한 번 기록된 데이터는 변경 불가
- **타입 안전**: Rust의 타입 시스템을 통한 직렬화/역직렬화

사용 예시:

```
// Guest에서 Journal에 데이터 기록
```

```
env::commit(&result);

env::commit(&metadata);



// Host에서 Journal 데이터 읽기

let result: u64 = receipt.journal.decode()?;

let metadata: String = receipt.journal.decode()?;
```

영지식 증명의 기본 원리와 RISC Zero에서의 적용

영지식 증명의 세 가지 속성

1. 완전성 (Completeness)

정의: 올바른 증명은 항상 검증을 통과해야 함

RISC Zero에서의 구현:

- 올바르게 실행된 Guest 프로그램은 항상 유효한 Receipt 생성
- 모든 RISC-V 명령어와 시스템 호출이 정확히 추적됨

2. 건전성 (Soundness)

정의: 잘못된 증명은 검증을 통과할 수 없어야 함

RISC Zero에서의 보장:

- 회로 제약 조건을 통한 실행 무결성 보장
- FRI 프로토콜의 암호학적 보안성 (98비트 보안 강도)
- Image ID를 통한 프로그램 무결성 검증

3. 영지식성 (Zero-Knowledge)

정의: 증명 자체로는 비밀 정보를 유추할 수 없어야 함

RISC Zero에서의 적용:

- 입력 데이터는 Receipt에 포함되지 않음
- Journal에는 명시적으로 commit된 데이터만 포함
- 실행 과정의 중간 상태는 암호학적으로 은닉

RISC Zero의 증명 생성 과정

1단계: 실행 추적 (Execution Trace)

```
// 각 명령어 실행마다 기록되는 정보
struct TraceRow {
cycle: u32, // 실행 사이클
pc: u32, // 프로그램 카운터
instruction: u32, // 실행된 명령어
registers: [u32; 32], // 레지스터 상태
memory_ops: Vec<MemoryOp>, // 메모리 연산
}
```

2단계: 대수적 중간 표현 (AIR - Algebraic Intermediate Representation)

- 실행 추적을 다항식 제약 조건으로 변환
- 각 RISC-V 명령어에 대한 제약 조건 정의
- 메모리 일관성 및 프로그램 흐름 제약 조건

3단계: zk-STARK 증명 생성

- FRI 프로토콜을 사용한 다항식 커밋먼트
- 50회의 쿼리를 통한 확률적 검증
- 약 200KB 크기의 STARK 증명 생성

4단계: 재귀적 압축 (선택적)

- STARK 증명을 Groth16으로 압축
- 최종 증명 크기: 약 200바이트
- 검증 시간 단축 및 온체인 검증 최적화

이러한 아키텍처를 통해 RISC Zero는 임의의 Rust 프로그램에 대한 영지식 증명을 효율적으로 생성하고 검증할 수 있는 강력한 플랫폼을 제공합니다.

다음 문서: Guest 프로그램 프로세스